Project usecase : Automobile Repair

Use case

This dataset represents a multi-location vehicle repair business where vehicles are repaired, billed, and delivered back to customers. The business tracks repair timelines, estimates, revenue, staff performance, and customer feedback across stores to analyse operational efficiency, financial performance, and customer experience over time.

Data Ingestion

The raw data is currently stored in Azure Blob Storage, and the ingestion pipeline is designed to extract this data using the Fivetran API. Since ourdata bricks free trial environment is provisioned on AWS Databricks, we are unable to generate or use Azure Blob Storage–specific credentials within this setup. Hence the Fivetran api is used.

Tables Provided

- Customer_survey
- Store
- Order
- Estimate
- Ns_budget
- Invoice

Layering

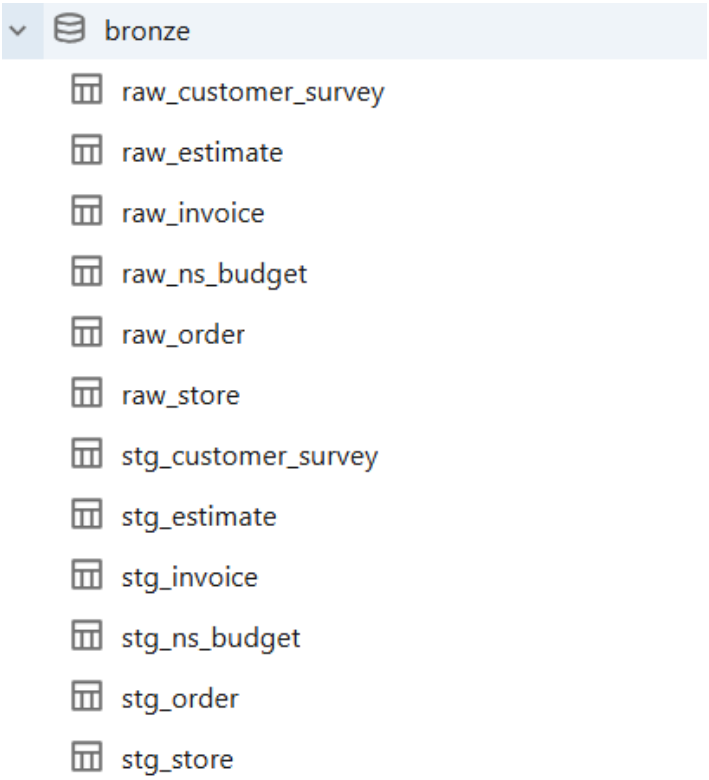
Bronze Layer

I have ingested raw data from Azure Blob Storage into the automobilerepair.bronze schema for the following tables: customer_survey, estimate, invoice, ns_budget, order, and store.

For each table, I created a raw table and then a staging table using SCD Type 1 logic, adding audit columns such as inserted_at and updated_at.

The staging tables are used to standardize and prepare the data for further processing.

Tables in Bronze Layer



Silver Layer

Data Cleaning & Transformation (SLV Layer)

1. **Basic data-quality steps performed:**
    - Handled null values.
    - Removed duplicate records.
    - Trimmed unnecessary whitespace.
    - Standardized and formatted date columns correctly.
2. **Null-value handling approach:**
    - Applied context-specific rules for each column.
    - Missing **manager names** in the *store* table were filled using **window functions**, leveraging the manager_id to propagate valid values.

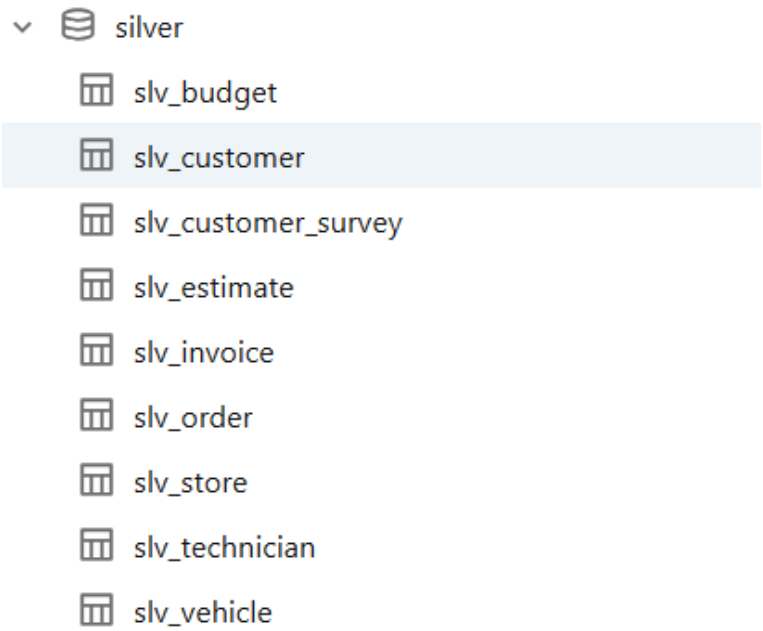
- For numerical fields like **estimates**, missing values were imputed using the **median** to maintain distribution integrity and avoid skewness.

3. **Technology used:**
    - All transformations were implemented using **PySpark**, enabling efficient processing of large-scale datasets.

Normalization of Orders Data

4. **Normalization performed on the orders table:**
    - Identified redundant and denormalized attributes.
    - Extracted and separated customer, technician, and vehicle information into dedicated **reference dimension tables**.
    - Achieved improved data integrity, reduced duplication, and enhanced query performance.

Tables in Silver Layer



Gold Layer

1. Star Schema Design

- The data model follows a Star Schema.
- In this structure, Fact tables store the main transactions.
- Dimension tables store descriptive details related to those transactions.

2. Fact Table Creation

- Data from orders, invoices, and estimates was combined.
- These were joined to form a single Fact table that contains all key business events.
- In your curated layer, these include tables like:
    - fact_orderlifecycle
    - fact_budge

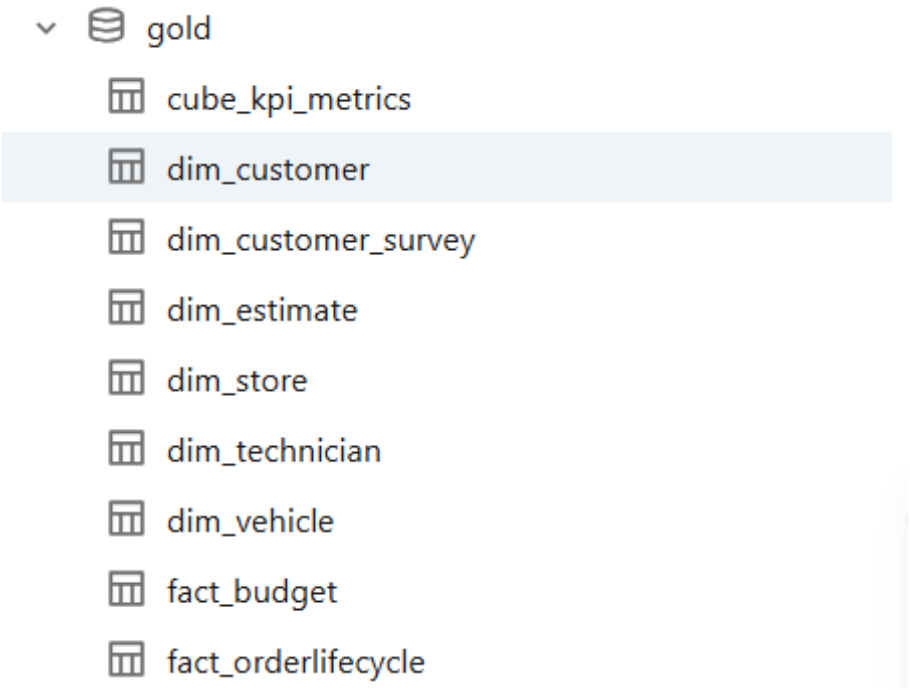
3. Dimension Tables

- Supporting information was separated into Dimension tables.
- These help in filtering, grouping, and analyzing data easily.
- Your curated dimensions include:
    - dim_customer
    - dim_customer_survey
    - dim_estimate
    - dim_store
    - dim_technician
    - dim_vehicle

4. Cube Table for KPIs

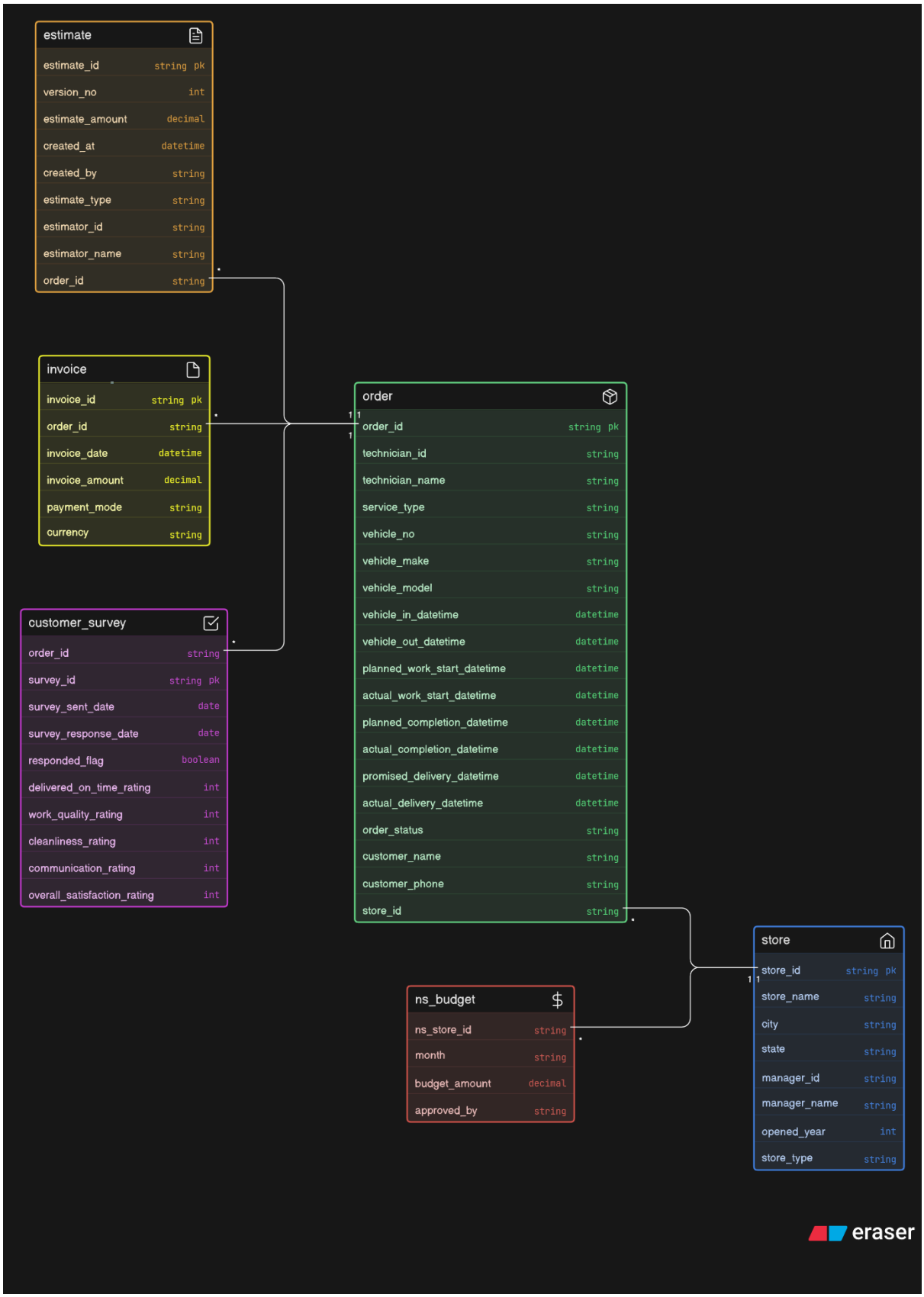
- A cube table named cube_kpi_metrics was created.
- It contains only the important fields needed for KPI calculations.
- This helps improve performance and makes dashboard queries faster

Tables in Gold Layer:



Data Modeling

Before Modelling



After Modelling

KPI

- MTD Performance vs Previous MTD
- Average Days in Shop
- Survey Coverage
- Survey Scores Summary
- Revenue vs Budget
- Top Technicians by Completion Time Accuracy
- Year-to-Date Revenue Growth
- Stage-wise Day Cycle Time
- Estimator Accuracy
- Technician Workload

Orchestration

- **Separate pipelines were created for each layer**, and each layer's processing is orchestrated individually.
- **A final master orchestration pipeline triggers all other pipelines** using the Run Job option, enabling end-to-end processing.
- **All jobs are scheduled with time limits**, and an email alert system is set up to notify the team if any pipeline fails.